

Step-by-Step Tutorial for an Automated GF180MCU Digital Design Workflow

Maryam Shahangian and James E. Stine
Oklahoma State University
Electrical and Computer Engineering Department
Stillwater, OK 74078 USA
{maryam.shahangian, james.stine}@okstate.edu

1 Introduction

The goal of this tutorial is to provide a simple, step-by-step workflow for running OpenROAD inside a containerized environment. The tutorial first introduces the basic concept of a container and explains why containerized environments are useful for digital design workflows. A container provides a ready-to-use environment in which the required design tools are already installed and configured. As a result, users do not need to manually install tools or resolve software dependency issues.

The tutorial then demonstrates how OpenROAD can automate the major stages of a digital design flow. Rather than executing each stage manually, OpenROAD-flow-scripts can guide the design from RTL input through synthesis, place-and-route, and final GDSII layout generation.

To demonstrate the workflow, the tutorial first uses an example design that is already included within OpenROAD. This provides a complete example of the design flow inside the container, beginning with the RTL design files and ending with the generated GDSII layout. Finally, the tutorial explains how the same flow can be adapted to use the **OSU GF180** standard-cell library.

2 Understanding the Environment

In this tutorial, the design environment is provided through a containerized setup. This means that the chip-design tools are not installed directly on the host operating system. Instead, they are provided inside a prepared environment that can be launched consistently across different machines. In practical terms, the host computer remains the user's normal system, while the actual design workflow runs inside a preconfigured software environment containing the required tools, libraries, scripts, and dependencies.

2.1 What IIC-OSIC-TOOLS Is

IIC-OSIC-TOOLS is the containerized tool environment used in this tutorial. It provides an integrated, ready-to-use platform for IC design, with the required tools already bundled together. This makes setup easier and avoids the need to install and configure each tool manually.

2.2 What Docker Does

Docker is the software used to launch and manage the prepared environment. In simple terms, the container is the workspace that already contains the design tools, while Docker is the software that creates, starts, and manages that workspace. In this tutorial, Docker is used to run the IIC-OSIC-TOOLS container so that the OpenROAD workflow can be executed in a consistent and reproducible environment.

3 Installing Docker

Docker Desktop installers are available for multiple operating systems and architectures. The official Docker download page provides installers for Windows, Linux, and macOS. In this tutorial, the documented setup was performed on Windows. Therefore, the following steps focus specifically on the Windows installation path and the continuation of the environment setup.

1. Download Docker Desktop from the official Docker website for the appropriate operating system.

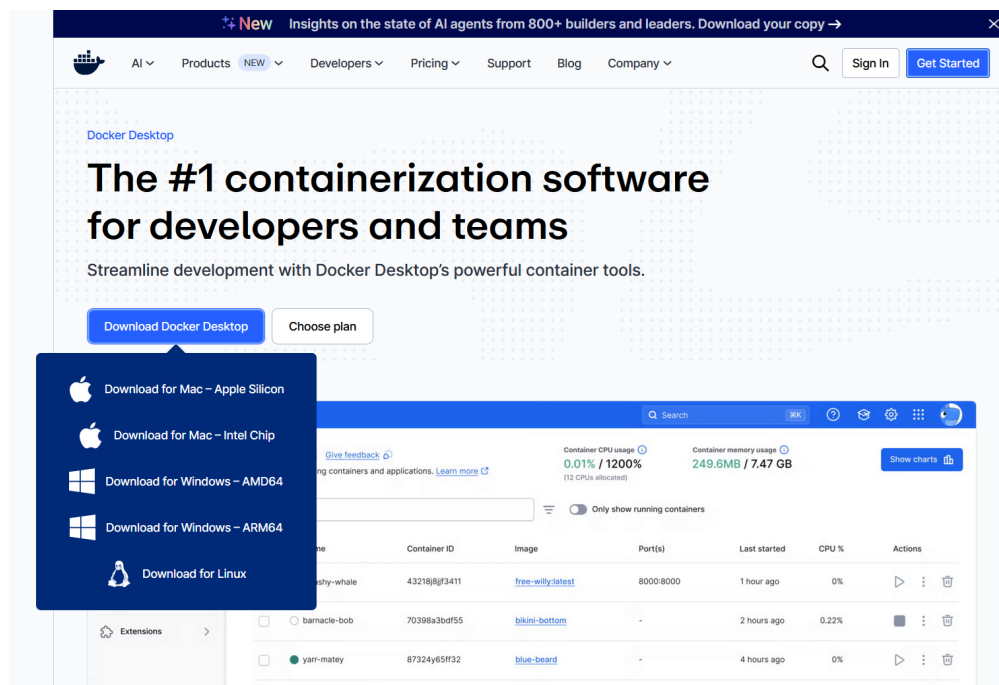


Figure 1: Docker Desktop download page for selecting the appropriate installer.

2. Follow the installation instructions provided by Docker Desktop to complete the installation on the host machine.

After Docker Desktop is installed, select the preferred method for accessing the container environment. Three access modes are available:

- **VNC Mode:** Provides a full graphical desktop environment that can be accessed through a web browser.

- **Jupyter Mode:** Provides a web-based notebook environment useful for running commands, documenting steps, and working interactively.
- **Shell Mode:** Provides a simple command-line terminal for running scripts and server-side tasks.

For this tutorial, Jupyter Mode is selected as the primary access method.

3.1 Select the Correct Chipathon Branch and Docker Image

After cloning the repository,

1. open Command Prompt (CMD) and run the following commands:

```
cd %USERPROFILE%\eda\designs\iic-osic-tools
```

```
git checkout ieee_chipathon
```

This switches the repository to the branch used for the Chipathon setup.

Note: `DESIGNS=$HOME/eda/designs` (`DESIGNS=%USERPROFILE%\eda\designs` for `.bat`) sets the directory that holds your design files. This directory is mounted into the container on `/foss/designs`.

2. Next, set the Docker image tag and start Jupyter:

```
SET DOCKER_TAG=chipathon26
start_jupyter.bat
```

These commands switch to the Chipathon branch and start the Jupyter container using the Chipathon 26 image.

3. If a Jupyter container was already created previously using an older image, remove it first and then start it again:

```
docker rm -f iic-osic-tools_jupyter
SET DOCKER_TAG=chipathon26
start_jupyter.bat
```

```
Command Prompt
37d42f817fff: Pull complete
4f378f4259ed: Pull complete
6d1152968bbe: Pull complete
c81169a9abc5: Pull complete
28e021382a6a: Pull complete
93eaa9e8631: Pull complete
1aa86d939986: Pull complete
4975c1a78d01: Pull complete
3b832a17a43d: Pull complete
f28d8dd31d6d: Pull complete
f415ea8233f3: Pull complete
c59ef558d12a: Pull complete
e7f96789d513: Pull complete
8d1e8d6a6393: Pull complete
5f1a8569ff71: Pull complete
18d86db375c8: Pull complete
fd9f3ef828ae: Pull complete
3867c4622843: Pull complete
3868d2c4831: Pull complete
d8b24a4d58c6: Download complete
Digest: sha256:1ba754709b0d71b11ce9357c7d2d63b6a4562c2a36b28c1dde33dd0979a9b4
Status: Downloaded newer image for hpret/iic-asic-tools:chipathon26
docker.io/hpret/iic-asic-tools:chipathon26

What's next:
View a summary of image vulnerabilities and recommendations → docker scout quickview hpret/iic-asic-tools:chipathon26
adfdac1a9f7a7b6723fb9942a835d6c7ef96f5e5346c76f78f2c826f1a992
C:\Users\Maryam\designs\iic-asic-tools>
```

Figure 2: Successful startup of the Chipathon 26 Jupyter container in Command Prompt.

4. After that, open Docker Desktop. Click the Play button next to `iic-asic-tools_jupyter` and wait a few seconds for the container to start. Once the container is running, go to the **Ports** column and click `8888:8888` to open the Jupyter environment in a web browser.

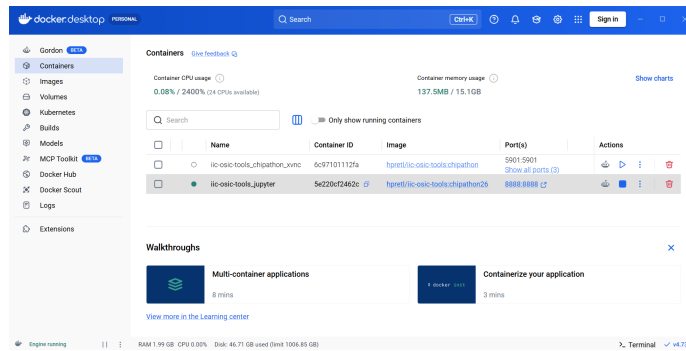


Figure 3: Docker Desktop showing the running Jupyter container and the available port link.

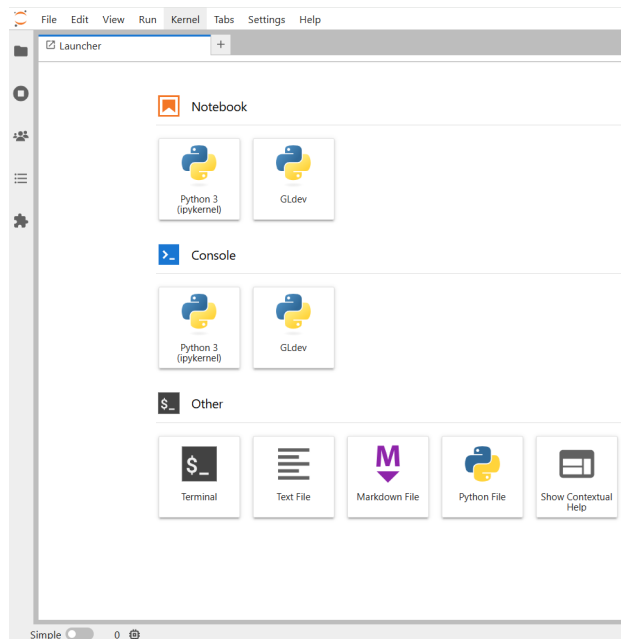


Figure 4: JupyterLab launcher page after the container starts successfully.

Tip: If the port number shown in Docker Desktop is not clickable, manually open a web browser and go to: `http://localhost:8888`

For VNC mode, click the corresponding VNC port link in Docker Desktop, or manually open: `http://localhost:80` When prompted, enter the default password: `abc123` At this point, the JupyterLab interface should become visible in the browser, indicating that the container environment has started successfully and is ready for interactive use. The primary shared working directories are:

- Windows: `C:\Users\<username>\eda\designs`
- Container: `/foss/designs`

Use this folder for all files and designs throughout the tutorial.

3.2 Check the Installed Tools

To open a terminal in JupyterLab, go to the **Launcher** page and, under **Other**, click **Terminal**. Then run the following commands:

```
cd /foss/designs && pwd && ls
```

To verify the installed tools and display the ORFS commit version, run:

```
yosys -V
```

```
openroad -version
```

```
echo $TOOLS
```

```
cat $TOOLS/openroad/ORFS_COMMIT
```

These commands confirm that the required tools are available inside the container and display the matching ORFS commit used by the environment.

What Is an “Image”?

In Docker terminology, an image is a prebuilt software environment that already contains the required tools and dependencies. In this tutorial, the **Chipathon26** image is used. This image is distributed through the IIC-OSIC-TOOLS environment and maintained by Harald Pretl and collaborators. It already includes the main tools required for the workflow, such as OpenROAD and Yosys.

Why is the commit important?

The ORFS commit helps confirm that the OpenROAD-flow-scripts version matches the OpenROAD tools installed inside the container. This avoids compatibility problems caused by version mismatches.

3.3 Clone OpenROAD-flow-scripts

Next, clone the [OpenROAD-flow-scripts](#) repository and check out the commit version stored in the container image.

```
cd /foss/designs
```

```
git clone https://github.com/The-OpenROAD-Project/OpenROAD-flow-scripts.git
```

```
cd OpenROAD-flow-scripts
```

```
git checkout $(cat $TOOLS/openroad/ORFS_COMMIT)
```

This ensures that the flow scripts match the OpenROAD tools provided by the container image.

3.3.1 Setting the Tool Paths

Now configure the tool paths so that the flow uses the OpenROAD and Yosys executables installed inside the container.

```
cat >> ~/.bashrc <<'EOF'
export YOSYS_EXE=/foss/tools/yosys/bin/yosys
export OPENROAD_EXE=/foss/tools/openroad/bin/openroad
export OPENSTA_EXE=/foss/tools/openroad/bin/sta
EOF
```

```
source ~/.bashrc
```

This configuration step only needs to be performed once. Afterward, the terminal session will automatically recognize these tool paths.

Note: In this setup, the flow scripts expect the executables to exist under the local `tools/install` directory. However, the Chipathon 26 container already provides these tools under `/foss/tools`.

The following one-time setup creates symbolic links to the installed tools so that the OpenROAD flow can start normally.

```
cd /foss/designs/OpenROAD-flow-scripts
mkdir -p tools/install
ln -sf /foss/tools/yosys tools/install/yosys
ln -sf /foss/tools/openroad tools/install/OpenROAD
```

3.3.2 Run the Flow

Move to the flow directory and run:

```
cd /foss/designs/OpenROAD-flow-scripts/flow
```

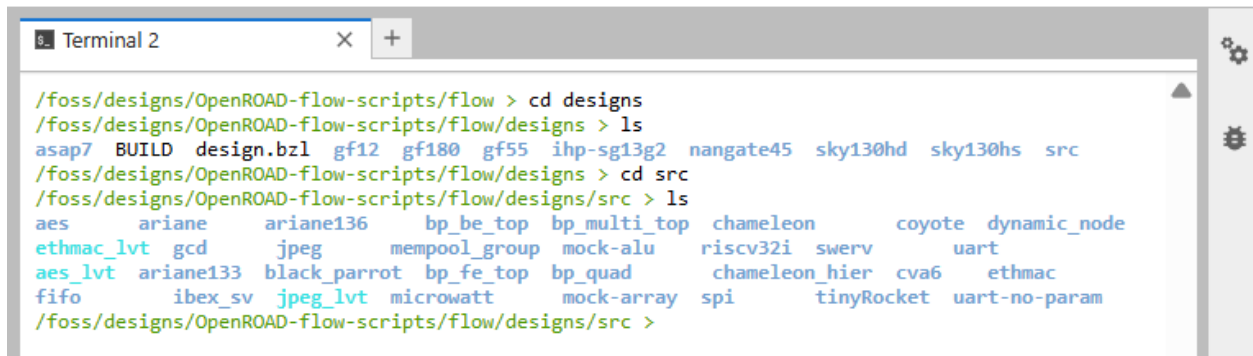
```
make
```

This starts the OpenROAD flow using the tool versions provided by the current Chipathon 26 container image. At this stage, the OpenROAD environment inside the IIC-OSIC-TOOLS container is fully configured and ready for use. The environment is now prepared so that the focus can shift to the main objective: running and understanding chip-design examples.

4 Run the AES Example Design

In this section, the OpenROAD flow is tested using a ready-to-use example design provided by OpenROAD. This helps verify that the environment is functioning correctly before moving to custom designs. The AES example is used here. The goal is not to modify the design, but to understand how the automated flow operates from RTL input to final layout generation. The RTL source files for the example are located inside the **designs/src** directory. The platform-specific configuration files for AES are stored under the corresponding GF180 design directory. First, move to the designs directory and enter the source folder:

```
cd /foss/designs/OpenROAD-flow-scripts/flow/designs
cd src
ls
```

A terminal window titled 'Terminal 2' showing a series of commands and their outputs. The user navigates to the directory /foss/designs/OpenROAD-flow-scripts/flow/designs and lists the contents, which include various design names like asap7, BUILD, design.bzl, gf12, gf180, gf55, ihp-sg13g2, nangate45, sky130hd, sky130hs, and src. Then, the user navigates to the src directory and lists its contents, showing a large number of files and subdirectories including aes, ariane, ariane136, bp_be_top, bp_multi_top, chameleon, coyote, dynamic_node, ethmac_lvt, gcd, jpeg, mempool_group, mock-alu, riscv32i, swerv, uart, aes_lvt, ariane133, black_parrot, bp_fe_top, bp_quad, chameleon_hier, cva6, ethmac, fifo, ibex_sv, jpeg_lvt, microwatt, mock-array, spi, tinyRocket, and uart-no-param.

```
/foss/designs/OpenROAD-flow-scripts/flow > cd designs
/foss/designs/OpenROAD-flow-scripts/flow/designs > ls
asap7  BUILD  design.bzl  gf12  gf180  gf55  ihp-sg13g2  nangate45  sky130hd  sky130hs  src
/foss/designs/OpenROAD-flow-scripts/flow/designs > cd src
/foss/designs/OpenROAD-flow-scripts/flow/designs/src > ls
aes      ariane    ariane136  bp_be_top  bp_multi_top  chameleon    coyote  dynamic_node
ethmac_lvt  gcd      jpeg      mempool_group  mock-alu    riscv32i    swerv    uart
aes_lvt  ariane133  black_parrot  bp_fe_top  bp_quad      chameleon_hier  cva6    ethmac
fifo      ibex_sv   jpeg_lvt   microwatt   mock-array   spi          tinyRocket  uart-no-param
/foss/designs/OpenROAD-flow-scripts/flow/designs/src >
```

Figure 5: Listing available example designs in OpenROAD-flow-scripts, including the AES design example.

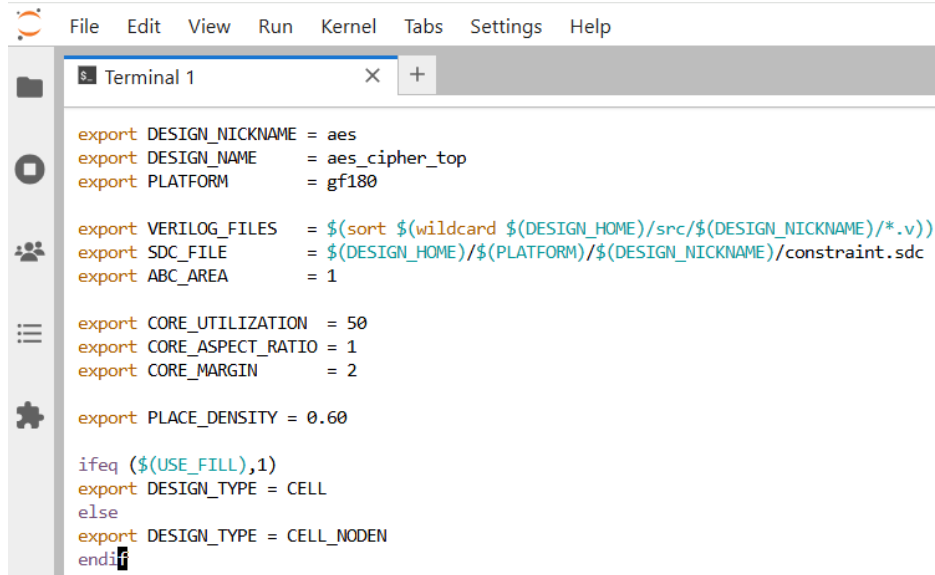
In this directory, the available example designs can be seen. One of them is **aes**. Next, move to the AES platform directory and list its files:

```
cd /foss/designs/OpenROAD-flow-scripts/flow/designs/gf180/aes
ls
```

The main configuration files for the AES example can now be seen, including:

- `config.mk`
- `constraint.sdc`

The **config.mk** file defines the primary design setup, while the **constraint.sdc** file contains the timing constraints used by the flow. Before running the AES example, it is useful to briefly inspect the **config.mk** file. This file contains the main configuration for the design, including the design name, target platform, Verilog source files, and timing constraints used during the flow.



```

export DESIGN_NICKNAME = aes
export DESIGN_NAME     = aes_cipher_top
export PLATFORM        = gf180

export VERILOG_FILES   = $(sort $(wildcard $(DESIGN_HOME)/src/$(DESIGN_NICKNAME)/*.v))
export SDC_FILE        = $(DESIGN_HOME)/$(PLATFORM)/$(DESIGN_NICKNAME)/constraint.sdc
export ABC_AREA        = 1

export CORE_UTILIZATION = 50
export CORE_ASPECT_RATIO = 1
export CORE_MARGIN      = 2

export PLACE_DENSITY = 0.60

ifeq ($(USE_FILL),1)
export DESIGN_TYPE = CELL
else
export DESIGN_TYPE = CELL_NODEN
endif

```

Figure 6: Example contents of the AES `config.mk` file used to define the design, platform, RTL sources, and implementation settings.

After that, move to the flow directory and run the AES example:

```
cd /foss/designs/OpenROAD-flow-scripts/flow
```

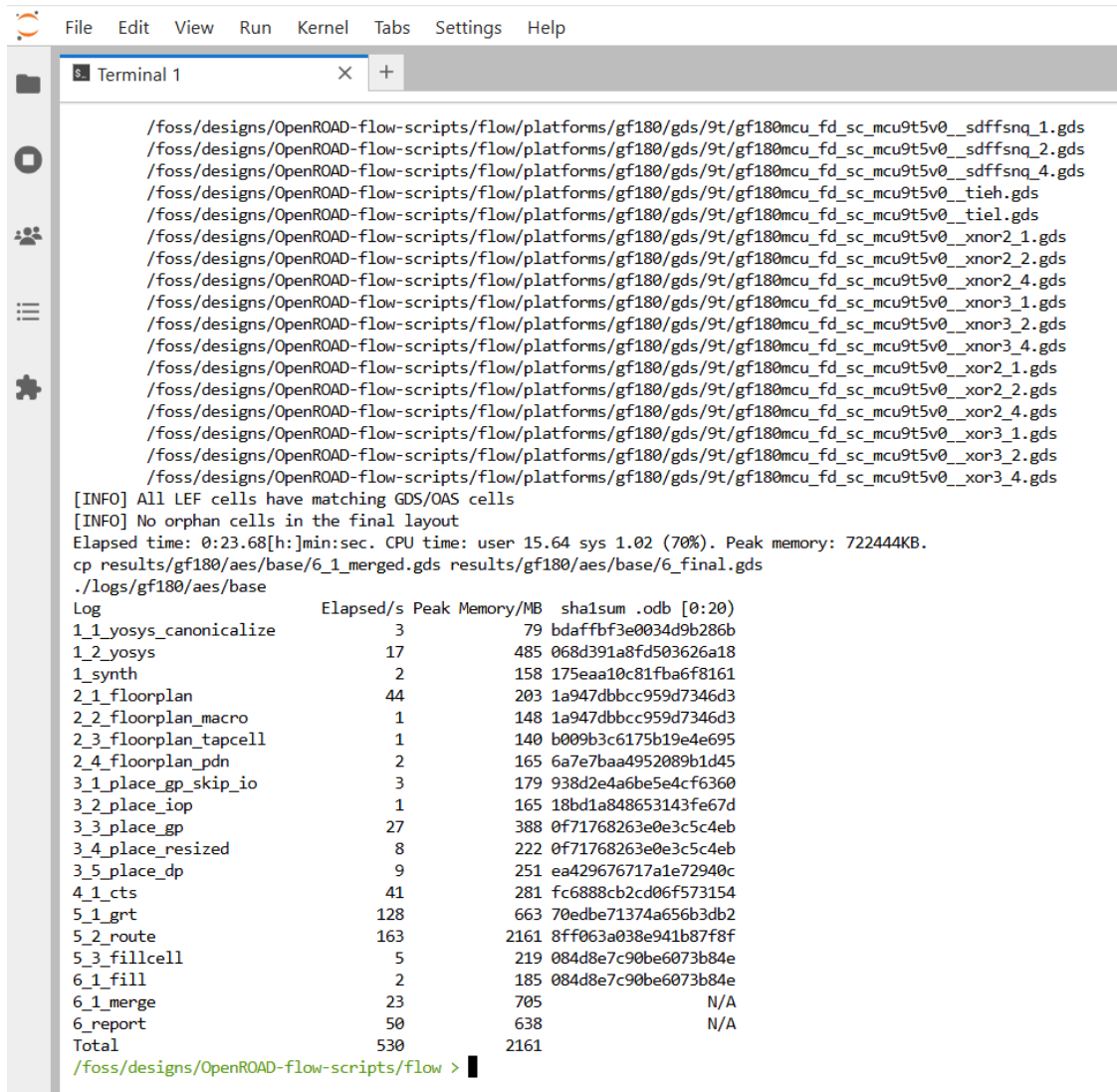
```
make DESIGN_CONFIG=./designs/gf180/aes/config.mk
```

As soon as this command is executed, the automated OpenROAD design flow begins. The first major stage is **synthesis**, where the RTL description is converted into a gate-level netlist. This step is performed by **Yosys**. After synthesis, the flow automatically continues through the remaining physical-design stages, including:

- Floorplanning
- Placement
- Clock Tree Synthesis (CTS)
- Routing

These stages are executed sequentially without requiring manual intervention. Depending on the complexity of the design and the available system resources, the complete flow may take several

minutes to finish.



```
File Edit View Run Kernel Tabs Settings Help

Terminal 1

/foss/designs/OpenROAD-flow-scripts/flow/platforms/gf180/gds/9t/gf180mcu_fd_sc_mcu9t5v0_sdfsnq_1.gds
/foss/designs/OpenROAD-flow-scripts/flow/platforms/gf180/gds/9t/gf180mcu_fd_sc_mcu9t5v0_sdfsnq_2.gds
/foss/designs/OpenROAD-flow-scripts/flow/platforms/gf180/gds/9t/gf180mcu_fd_sc_mcu9t5v0_sdfsnq_4.gds
/foss/designs/OpenROAD-flow-scripts/flow/platforms/gf180/gds/9t/gf180mcu_fd_sc_mcu9t5v0_tieh.gds
/foss/designs/OpenROAD-flow-scripts/flow/platforms/gf180/gds/9t/gf180mcu_fd_sc_mcu9t5v0_tiel.gds
/foss/designs/OpenROAD-flow-scripts/flow/platforms/gf180/gds/9t/gf180mcu_fd_sc_mcu9t5v0_xnor2_1.gds
/foss/designs/OpenROAD-flow-scripts/flow/platforms/gf180/gds/9t/gf180mcu_fd_sc_mcu9t5v0_xnor2_2.gds
/foss/designs/OpenROAD-flow-scripts/flow/platforms/gf180/gds/9t/gf180mcu_fd_sc_mcu9t5v0_xnor2_4.gds
/foss/designs/OpenROAD-flow-scripts/flow/platforms/gf180/gds/9t/gf180mcu_fd_sc_mcu9t5v0_xnor3_1.gds
/foss/designs/OpenROAD-flow-scripts/flow/platforms/gf180/gds/9t/gf180mcu_fd_sc_mcu9t5v0_xnor3_2.gds
/foss/designs/OpenROAD-flow-scripts/flow/platforms/gf180/gds/9t/gf180mcu_fd_sc_mcu9t5v0_xnor3_4.gds
/foss/designs/OpenROAD-flow-scripts/flow/platforms/gf180/gds/9t/gf180mcu_fd_sc_mcu9t5v0_xor2_1.gds
/foss/designs/OpenROAD-flow-scripts/flow/platforms/gf180/gds/9t/gf180mcu_fd_sc_mcu9t5v0_xor2_2.gds
/foss/designs/OpenROAD-flow-scripts/flow/platforms/gf180/gds/9t/gf180mcu_fd_sc_mcu9t5v0_xor2_4.gds
/foss/designs/OpenROAD-flow-scripts/flow/platforms/gf180/gds/9t/gf180mcu_fd_sc_mcu9t5v0_xor3_1.gds
/foss/designs/OpenROAD-flow-scripts/flow/platforms/gf180/gds/9t/gf180mcu_fd_sc_mcu9t5v0_xor3_2.gds
/foss/designs/OpenROAD-flow-scripts/flow/platforms/gf180/gds/9t/gf180mcu_fd_sc_mcu9t5v0_xor3_4.gds

[INFO] All LEF cells have matching GDS/OAS cells
[INFO] No orphan cells in the final layout
Elapsed time: 0:23.68[h:]min:sec. CPU time: user 15.64 sys 1.02 (70%). Peak memory: 722444KB.
cp results/gf180/aes/base/6_1_merged.gds results/gf180/aes/base/6_final.gds
./logs/gf180/aes/base
Log
Elapsed/s Peak Memory/MB sha1sum .odb [0:20)
1_1_yosys_canonicalize 3 79 bdaffbf3e0034d9b286b
1_2_yosys 17 485 068d391a8fd503626a18
1_synth 2 158 175eaa10c81fba6f8161
2_1_floorplan 44 203 1a947dbbcc959d7346d3
2_2_floorplan_macro 1 148 1a947dbbcc959d7346d3
2_3_floorplan_tapcell 1 140 b009b3c6175b19e4e695
2_4_floorplan_pdn 2 165 6a7e7baa4952089b1d45
3_1_place_gp_skip_io 3 179 938d2e4a6be5e4c6f360
3_2_place_iop 1 165 18bd1a848653143fe67d
3_3_place_gp 27 388 0f71768263e0e3c5c4eb
3_4_place_resized 8 222 0f71768263e0e3c5c4eb
3_5_place_dp 9 251 ea429676717a1e72940c
4_1_cts 41 281 fc6888cb2cd06f573154
5_1_grt 128 663 70edbe71374a656b3db2
5_2_route 163 2161 8ff063a038e941b87f8f
5_3_fillcell 5 219 084d8e7c90be6073b84e
6_1_fill 2 185 084d8e7c90be6073b84e
6_1_merge 23 705 N/A
6_report 50 638 N/A
Total 530 2161
/foss/designs/OpenROAD-flow-scripts/flow > █
```

Figure 7: Successful completion of the automated OpenROAD AES implementation flow, including synthesis, placement, CTS, routing, and final GDS generation.

In Jupyter mode, the graphical interface may not appear because Jupyter provides a terminal-oriented environment rather than a full desktop interface. To view the final layout graphically, either use the VNC mode or open the generated GDS file locally using KLayout. A simple way to access the final layout is to download the generated GDS file directly from the JupyterLab file browser. The generated GDS file is typically located at:

```
/foss/designs/OpenROAD-flow-scripts/flow/results/gf180/aes/base/6_1_merged.gds
```

After downloading the file to the local computer, open it using **KLayout**.

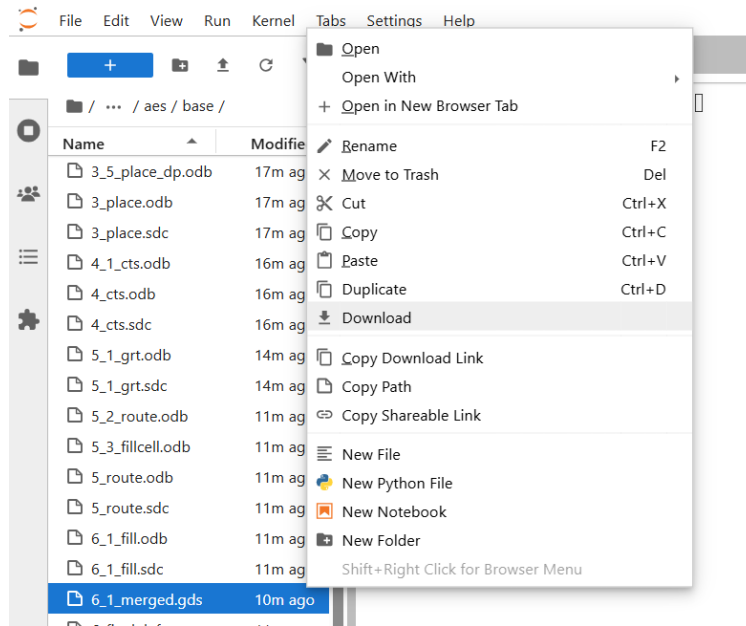


Figure 8: Downloading the generated merged GDS file from JupyterLab and opening it locally with KLayout.

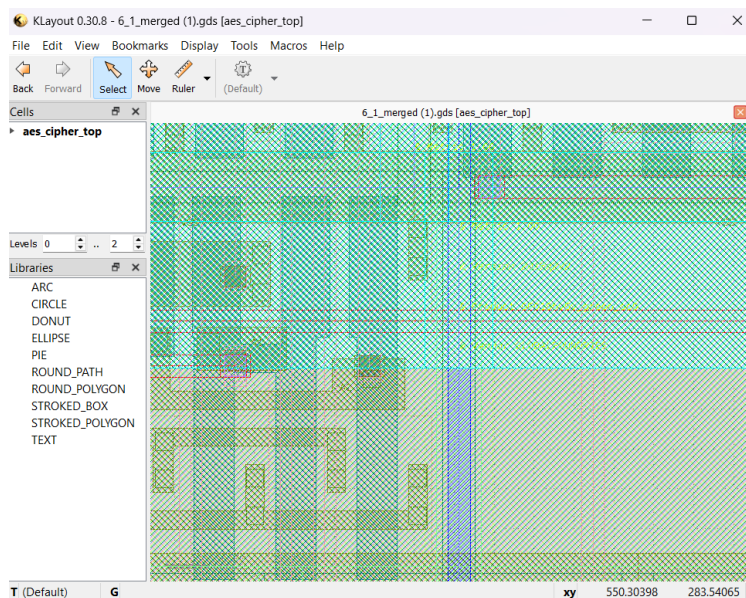


Figure 9: Viewing the final AES GDS layout in KLayout after completing the automated OpenROAD flow.

5 Worked Example: AES Implementation with the GF180MCU OSU Standard-Cell Library

In the previous section, the built-in AES example provided by OpenROAD-flow-scripts was used to demonstrate the default automated implementation flow inside the container environment. That example served as a baseline for understanding how the flow operates using the standard design setup already available in OpenROAD.

In this section, the same AES design is reused, but the library environment is modified so that the design is implemented with the GF180MCU OSU standard-cell library instead of the default library configuration. The purpose of this worked example is to show how an existing OpenROAD example can be adapted to a different standard-cell library and then evaluated within the same containerized flow.

This approach is useful because it allows the effect of library substitution to be studied on a realistic design rather than on a minimal test case. Since AES is already available in the OpenROAD example set, it provides a convenient and meaningful benchmark for documenting the required configuration changes, the integration procedure, and the practical issues that arise during implementation.

The standard cells used in this work are taken from the GF180MCU OSU standard-cell library developed at Oklahoma State University. This library was designed by James E. Stine, Jr. and the VLSI Computer Architecture Research Group.

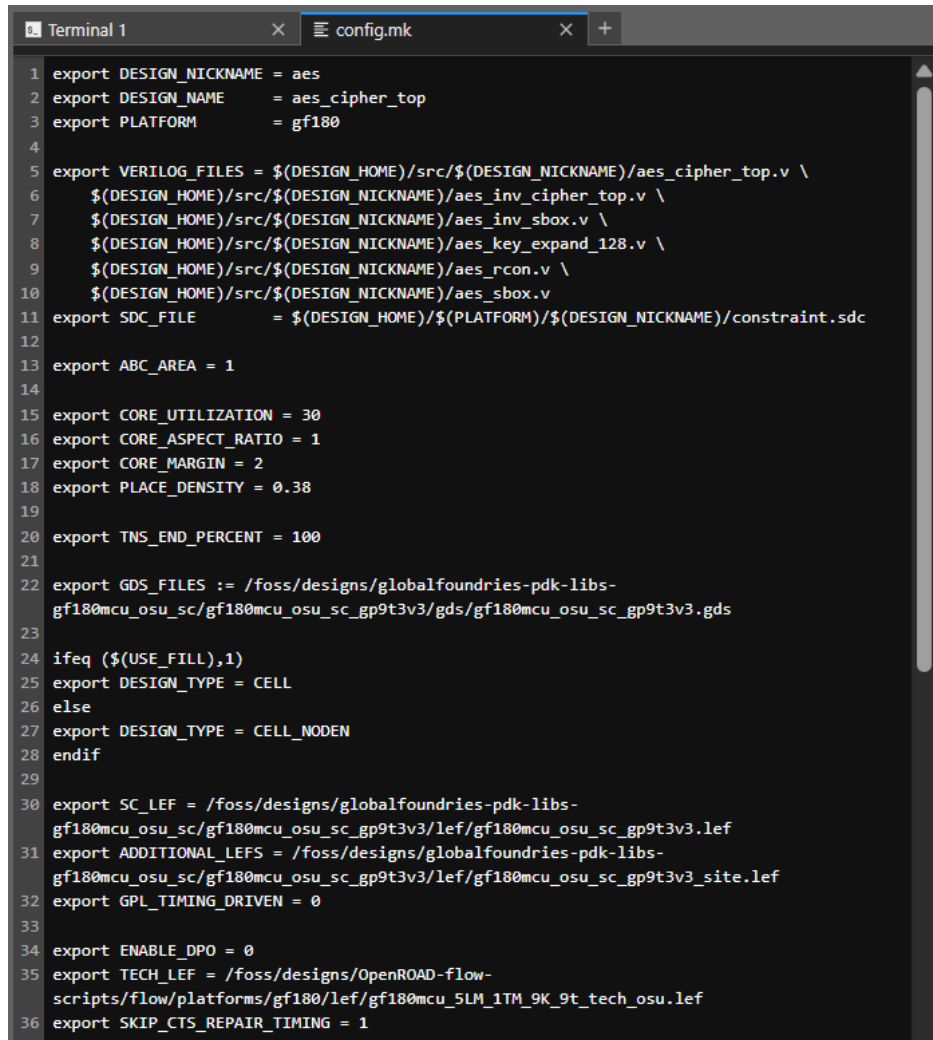
5.1 Matching the OpenROAD Configuration to the OSU Library

The AES RTL files and the related configuration files have to be organized so that they can be referenced consistently by the modified design setup. This step is important because the OpenROAD flow depends on a fixed directory structure and correctly referenced input files in order to recognize and process the design properly.

To use the GF180MCU OSU standard-cell library instead of the default library, the relevant OpenROAD configuration files must be updated in the proper locations. In this setup, the design-specific configuration is modified in the AES design directory, while the platform-level settings are adjusted in the GF180 platform directory.

The main design-level changes are made in the following file:

- `/foss/designs/OpenROAD-flow-scripts/flow/designs/gf180/aes/config.mk`



```
1 export DESIGN_NICKNAME = aes
2 export DESIGN_NAME     = aes_cipher_top
3 export PLATFORM         = gf180
4
5 export VERILOG_FILES = $(DESIGN_HOME)/src/$(DESIGN_NICKNAME)/aes_cipher_top.v \
6   $(DESIGN_HOME)/src/$(DESIGN_NICKNAME)/aes_inv_cipher_top.v \
7   $(DESIGN_HOME)/src/$(DESIGN_NICKNAME)/aes_inv_sbox.v \
8   $(DESIGN_HOME)/src/$(DESIGN_NICKNAME)/aes_key_expand_128.v \
9   $(DESIGN_HOME)/src/$(DESIGN_NICKNAME)/aes_rcon.v \
10  $(DESIGN_HOME)/src/$(DESIGN_NICKNAME)/aes_sbox.v
11 export SDC_FILE        = $(DESIGN_HOME)/$(PLATFORM)/$(DESIGN_NICKNAME)/constraint.sdc
12
13 export ABC_AREA = 1
14
15 export CORE_UTILIZATION = 30
16 export CORE_ASPECT_RATIO = 1
17 export CORE_MARGIN = 2
18 export PLACE_DENSITY = 0.38
19
20 export TNS_END_PERCENT = 100
21
22 export GDS_FILES := /foss/designs/globalfoundries-pdk-libs-
23   gf180mcu_osu_sc/gf180mcu_osu_sc_gp9t3v3/gds/gf180mcu_osu_sc_gp9t3v3.gds
24
25 ifeq ($(USE_FILL),1)
26 export DESIGN_TYPE = CELL
27 else
28 export DESIGN_TYPE = CELL_NODEN
29 endif
30
31 export SC_LEF = /foss/designs/globalfoundries-pdk-libs-
32   gf180mcu_osu_sc/gf180mcu_osu_sc_gp9t3v3/lef/gf180mcu_osu_sc_gp9t3v3.lef
33
34 export ADDITIONAL_LEFS = /foss/designs/globalfoundries-pdk-libs-
35   gf180mcu_osu_sc/gf180mcu_osu_sc_gp9t3v3/lef/gf180mcu_osu_sc_gp9t3v3_site.lef
36
37 export GPL_TIMING_DRIVEN = 0
38
39 export ENABLE_DPO = 0
40
41 export TECH_LEF = /foss/designs/OpenROAD-flow-
42   scripts/flow/platforms/gf180/lef/gf180mcu_5LM_1TM_9K_9t_tech_osu.lef
43
44 export SKIP_CTS_REPAIR_TIMING = 1
```

Figure 10: Modified AES config.mk file located under flow/designs/gf180/aes.

This file is updated so that the AES design points to the modified synthesized netlist, the OSU GDS file, the OSU standard-cell LEF, the OSU site LEF, and the technology LEF used for this library setup. In this way, the design is evaluated using the intended OSU library views rather than the default GF180 library configuration.

The platform-level changes are applied in:

- /foss/designs/OpenROAD-flow-scripts/flow/platforms/gf180/config.mk

```

Terminal 3  config.mk
1 #/OpenROAD-flow-scripts/flow/platforms/gf180/config.mk
2 #-----
3 # Tech/Libs
4 #   TRACK_OPTION = 9t
5 #   METAL_OPTION = 5LM_1TM
6 #-----
7 export TRACK_OPTION    ?= 9t
8 export METAL_OPTION    ?= 5LM_1TM
9 export KVALUE           ?= 9
10 export POWER_OPTION    ?= 5v0
11 export CORNER          ?= BC
12 export PROCESS         = 180
13 #-----
14 # OpenROAD
15 #-----
16 export TECH_LEF         ?= $(PLATFORM_DIR)/lef/gf180mcu_$(METAL_OPTION)_$(KVALUE)K_$(TRACK_OPTION)_tech.lef
17 export SC_LEF           ?= $(PLATFORM_DIR)/lef/gf180mcu_$(METAL_OPTION)_$(KVALUE)K_$(TRACK_OPTION)_sc.lef
18
19 export GDS_FILES        ?= $(wildcard $(PLATFORM_DIR)/gds/$(TRACK_OPTION)/*.gds)
20 export GDS_FILES        += $(ADDITIONAL_GDS)
21 # Dont use cells
22 export DONT_USE_CELLS   :=
23
24 # Fill cells used in fill cell insertion
25 export FILL_CELLS       ?= gf180mcu_osu_sc_gp9t3v3_fill_16 \
26                           gf180mcu_osu_sc_gp9t3v3_fill_8 \
27                           gf180mcu_osu_sc_gp9t3v3_fill_4 \
28                           gf180mcu_osu_sc_gp9t3v3_fill_2 \
29                           gf180mcu_osu_sc_gp9t3v3_fill_1
30
31 export TIE_CELL         = gf180mcu_osu_sc_gp9t3v3_tieh
32 export ENDCAP_CELL     :=
33 export RC_FILE          = $(PLATFORM_DIR)/setRC.tcl
34 #-----
35 # Yosys
36 #-----
37 # set the TIEHI/TIELO cells
38 export TIEHI_CELL_AND_PORT = gf180mcu_osu_sc_gp9t3v3_tieh Y
39 export TIELO_CELL_AND_PORT = gf180mcu_osu_sc_gp9t3v3_tiel Y
40 export ABC_DRIVER_CELL = gf180mcu_osu_sc_gp9t3v3_buf_4
41 export ABC_LOAD_IN_FF    = 0.01343
42
43 export MIN_BUF_CELL_AND_PORTS = gf180mcu_osu_sc_gp9t3v3_buf_1 A Y
44 export MAX_FANOUT        = 20
45
46 # Yosys mapping files
47 export LATCH_MAP_FILE    ?= $(PLATFORM_DIR)/cells_latch.v
48 export ADDER_MAP_FILE    ?= $(PLATFORM_DIR)/cells_adders.v

```

```

49 #-----
50 # Floorplan
51 #-----
52 export PLACE_SITE       = gf180mcu_osu_sc_gp9t3v3
53
54 # IO Placer pin layers
55 export IO_PLACER_H      ?= Metal3
56 export IO_PLACER_V      ?= Metal4
57
58 # Define default PDN config
59 export PDN_TCL           ?= $(PLATFORM_DIR)/openROAD/pdn/pdn_grid_strategy_$(TRACK_OPTION)_gh.cfg
60 export TAPCELL_TCL :=
61
62 # macro planning
63 export MACRO_PLACE_HALO ?= 10 10
64 #-----
65 # Place
66 #-----
67 export PLACE_DENSITY    ?= 0.40
68 #-----
69 # Route
70 #-----
71 # FastRoute options
72 export MIN_ROUTING_LAYER ?= Metal2
73 export MAX_ROUTING_LAYER ?= Metal5
74 export DISABLE_VIA_GEN  ?= 1
75
76 # Define fastRoute tcl
77 export FASTROUTE_TCL    ?= $(PLATFORM_DIR)/fastroute.tcl
78 # Klayout layer properties
79 export KLAYOUTH_TECH_FILE = $(PLATFORM_DIR)/Klayout/gf180mcu_$(METAL_OPTION)_$(KVALUE)K_$(TRACK_OPTION).lyt
80 export KLAYOUTH_LEF_FILE  = $(PLATFORM_DIR)/Klayout/tech_$(TRACK_OPTION)_$(METAL_OPTION).lef
81 export GDS_LAYER_MAP     = $(abspath
82 $(PLATFORM_DIR)/gds/$(TRACK_OPTION)/gf180mcu_$(METAL_OPTION)_$(KVALUE)K_$(TRACK_OPTION)_edizgds.layermap)
83
84 # For RCX
85 export BC_RCX_RULES      = $(PLATFORM_DIR)/openROAD/rcx/gf180mcu_1p5m_1tm_9k_sp_smin_OPTD_bst.rules
86 export WC_RCX_RULES      = $(PLATFORM_DIR)/openROAD/rcx/gf180mcu_1p5m_1tm_9k_sp_smin_OPTD_wst.rules
87 export TC_RCX_RULES      = $(PLATFORM_DIR)/openROAD/rcx/gf180mcu_1p5m_1tm_9k_sp_smin_OPTD_typ.rules
88
89 export BC_RCX_RC_CORNER  = FuncRCmin
90 export WC_RCX_RC_CORNER  = FuncRCmax
91 export TC_RCX_RC_CORNER  = FuncRCtyp
92
93 export RCX_RULES         = $($(CORNER)_RCX_RULES)
94 export RCX_RC_CORNER     = $($(CORNER)_RCX_RC_CORNER)

```

```

94 #-----
95 # define libraries - corner dependant setting
96 #-----
97
98 # standard cell section
99
100 export BC_LIB_FILES      = /foss/designs/globalfoundries-pdk-libs-
gf180mcu_osu_sc/gf180mcu_osu_sc_gp9t3v3/lib/gf180mcu_osu_sc_gp9t3v3_TT_25C.ccs.lib
101 export BC_TEMPERATURE    = 25C
102 export BC_VOLTAGE        = 5.0
103
104 export WC_LIB_FILES      = /foss/designs/globalfoundries-pdk-libs-
gf180mcu_osu_sc/gf180mcu_osu_sc_gp9t3v3/lib/gf180mcu_osu_sc_gp9t3v3_TT_25C.ccs.lib
105 export WC_TEMPERATURE    = 25C
106 export WC_VOLTAGE        = 5.0
107
108 export TC_LIB_FILES      = /foss/designs/globalfoundries-pdk-libs-
gf180mcu_osu_sc/gf180mcu_osu_sc_gp9t3v3/lib/gf180mcu_osu_sc_gp9t3v3_TT_25C.ccs.lib
109 export TC_TEMPERATURE    = 25C
110 export TC_VOLTAGE        = 5.0
111
112 #-----
113 # now, set files from user setting CORNER
114 #-----
115
116 export TEMPERATURE       = ${$(CORNER)_TEMPERATURE}
117 export VOLTAGE            = ${$(CORNER)_VOLTAGE}
118 export LIB_FILES          = ${$(CORNER)_LIB_FILES} \
119                             $(ADDITIONAL_LIBS)
120
121 # IR drop estimation supply net name to be analyzed and supply voltage variable
122 export PMR_NETS_VOLTAGES ?= VDD $(VOLTAGE)
123 export GND_NETS_VOLTAGES ?= VSS 0.0
124 export IR_DROP_LAYER     ?= Metall
125
126 # For proprietary tool enablements that are not public
127 export GF180_PRIVATE_DIR ?= ../../gf180-private
128 -include $(GF180_PRIVATE_DIR)/private.mk
129

```

Figure 11: GF180 platform config.mk sections used for the GF180MCU OSU library integration.

This file defines the platform defaults used by the OpenROAD flow, including library-dependent settings such as filler cells, tie cells, and technology-level references. These settings must be checked carefully to ensure that they remain consistent with the selected OSU library.

In addition to the configuration files, the technology-mapping files must also be matched to the target library. In this example, the files `cells_adders.v` and `cells_latch.v`, located under `/foss/designs/OpenROAD-flow-scripts/flow/platforms/gf180/`, are reviewed and aligned with the GF180MCU OSU library.

This means that the mapped cells, instance names, and pin connections used in these files must correspond exactly to the cell names and pin names provided by the OSU library.

```

Terminal 1
cells_adders.v

1 (* techmap_celltype = "$fa" *)
2 module _tech_fa (A, B, C, X, Y);
3     parameter WIDTH = 1;
4     (* force_downto *)
5     input [WIDTH-1 : 0] A, B, C;
6     (* force_downto *)
7     output [WIDTH-1 : 0] X, Y;
8     parameter _TECHMAP_CONSTVAL_A_ = WIDTH'bx;
9     parameter _TECHMAP_CONSTVAL_B_ = WIDTH'bx;
10    parameter _TECHMAP_CONSTVAL_C_ = WIDTH'bx;
11    genvar i;
12    generate for (i = 0; i < WIDTH; i = i + 1) begin
13        if (_TECHMAP_CONSTVAL_A_[i] == 1'b0 || _TECHMAP_CONSTVAL_B_[i]
14        == 1'b0 || _TECHMAP_CONSTVAL_C_[i] == 1'b0) begin
15            gf180mcu_osu_sc_gp9t3v3_addh_1 halfadder_Cconst (
16                .A(A[i]),
17                .B(B[i]),
18                .CO(X[i]), .SUM(Y[i])
19            );
20        end
21        else begin
22            if (_TECHMAP_CONSTVAL_B_[i] == 1'b0) begin
23                gf180mcu_osu_sc_gp9t3v3_addh_1 halfadder_Bconst (
24                    .A(A[i]),
25                    .B(C[i]),
26                    .CO(X[i]), .SUM(Y[i])
27                );
28            end
29            else begin
30                gf180mcu_osu_sc_gp9t3v3_addh_1 halfadder_Aconst (
31                    .A(B[i]),
32                    .B(C[i]),
33                    .CO(X[i]), .SUM(Y[i])
34                );
35            end
36        end
37    end
38    else begin
39        gf180mcu_osu_sc_gp9t3v3_addf_1 fulladder (
40            .A(A[i]), .B(B[i]), .CI(C[i]), .CO(X[i]), .SUM(Y[i])
41        );
42    end
43    end endgenerate
44 endmodule
45

```

(a) cells_adders.v

```

Terminal 3
cells_latch.v

1 module $_DLATCH_P(input E, input D, output Q);
2     gf180mcu_osu_sc_gp9t3v3_dlat_1 _TECHMAP_REPLACE_ (
3         .D(D),
4         .CLK(E),
5         .Q(Q)
6     );
7 endmodule
8
9 module $_DLATCH_N(input E, input D, output Q);
10    wire E_n;
11    assign E_n = ~E;
12
13    gf180mcu_osu_sc_gp9t3v3_dlatn_1 _TECHMAP_REPLACE_ (
14        .D(D),
15        .CLK(E_n),
16        .Q(Q)
17    );
18 endmodule
19

```

(b) cells_latch.v

Figure 12: Technology-mapping files aligned with the GF180MCU OSU standard-cell library.

Therefore, integrating the AES example with the GF180MCU OSU standard-cell library requires both configuration-level changes and cell-mapping consistency checks. The design files, platform files, and mapping files must all refer to the same library views in order for the OpenROAD flow to process the design correctly.

After the configuration is completed, go to the flow directory and run:

```
cd /foss/designs/OpenROAD-flow-scripts/flow
make DESIGN_CONFIG=./designs/gf180/aes/config.mk
```

If an error occurs during the flow, first fix the problem. Then remove the generated intermediate files and run the flow again from the same directory.

```
cd /foss/designs/OpenROAD-flow-scripts/flow
make clean_all DESIGN_CONFIG=./designs/gf180/aes/config.mk
make DESIGN_CONFIG=./designs/gf180/aes/config.mk
```

If the flow completes successfully, the final result files should be visible in the AES output directory. In particular, files such as `6_final.gds`, `6_final.def`, `6_final.v`, and `6_final.sdc` should appear. This confirms that the main OpenROAD implementation stages have completed correctly with the GF180MCU OSU standard-cell library. An example of the expected final output is shown in Figure 13. If the flow completes successfully, the final result files should be visible in the AES output directory. In particular, files such as `6_final.gds`, `6_final.def`, `6_final.v`, and `6_final.sdc` should appear. This confirms that the main OpenROAD implementation stages have completed correctly with the GF180MCU OSU standard-cell library.

An example of the expected final output is shown in Figure 13.


```

Terminal 1
[INFO] No orphan cells in the final layout
Elapsed time: 0:12.05[h]:min:sec. CPU time: user 11.68 sys 0.90 (104%). Peak memory: 692936KB.
cp results/gf180/aes/base/6_1_merged.gds results/gf180/aes/base/6_final.gds
./logs/gf180/aes/base
Log
Elapsed/s Peak Memory/MB sha1sum result [0:20)
1_1_yosys_canonicalize 2 56 ef1ca0e81d85abb143e0
1_2_yosys 25 483 0c975ed6599122fb9874
1_synth 2 132 6f6cd13089a7745c570b
2_1_floorplan 5 173 693e0f05e389e6c85d72
2_2_floorplan_macro 2 121 693e0f05e389e6c85d72
2_3_floorplan_tapcell 2 115 693e0f05e389e6c85d72
2_4_floorplan_pdn 5 139 565ace11eb503ef7d816
3_1_place_gp_skip_io 14 150 3394a3314bb4bdc88f75
3_2_place_iop 5 138 51decce7551970050b17
3_3_place_gp 41 284 afe17049112692de83a5
3_4_place_resized 16 179 afe17049112692de83a5
3_5_place_dp 18 208 314b26c6eb227845c551
4_1_cts 22 247 6dbde28a5155527329ea
5_1_grt 41 551 73aa09720f0236834178
5_2_route 231 2183 131d788dc02538553e8
5_3_fillcell 2 88 131d788dc02538553e8
6_1_fill 7 151 131d788dc02538553e8
6_1_merge 12 676 N/A
6_report 68 498 N/A
Total 520 2183
/foss/designs/OpenROAD-flow-scripts/flow > ls -lh /foss/designs/OpenROAD-flow-scripts/flow/results/gf180/aes/base/6_final.*
-rw-r--r-- 1 designer designers 37M May 18 17:25 /foss/designs/OpenROAD-flow-scripts/flow/results/gf180/aes/base/6_final.def
-rw-r--r-- 1 designer designers 31M May 18 17:27 /foss/designs/OpenROAD-flow-scripts/flow/results/gf180/aes/base/6_final.gds
-rw-r--r-- 1 designer designers 30M May 18 17:25 /foss/designs/OpenROAD-flow-scripts/flow/results/gf180/aes/base/6_final.odg
-rw-r--r-- 1 designer designers 34K May 18 17:26 /foss/designs/OpenROAD-flow-scripts/flow/results/gf180/aes/base/6_final.sdc
-rw-r--r-- 1 designer designers 15M May 18 17:26 /foss/designs/OpenROAD-flow-scripts/flow/results/gf180/aes/base/6_final.spef
-rw-r--r-- 1 designer designers 1.5M May 18 17:26 /foss/designs/OpenROAD-flow-scripts/flow/results/gf180/aes/base/6_final.v
/foss/designs/OpenROAD-flow-scripts/flow > cat /foss/designs/OpenROAD-flow-scripts/flow/reports/gf180/aes/base/metadata-base-o
k.json 2>/dev/null | python3 -m json.tool | head -50
Expecting value: line 1 column 1 (char 0)
/foss/designs/OpenROAD-flow-scripts/flow > grep -E "Design area|Cell type|Total" /foss/designs/OpenROAD-flow-scripts/flow/logs
/gf180/aes/base/6_report.log
Total power : 6.02e-01 W
Total power : 6.02e-01 W
Cell type report:
Total Count Area
13924 391030.46
Design area 391030 um^2 31% utilization.
/foss/designs/OpenROAD-flow-scripts/flow > grep -E "wns|tns|slack|worst" /foss/designs/OpenROAD-flow-scripts/flow/logs/gf180/a
es/base/6_report.log
/foss/designs/OpenROAD-flow-scripts/flow > grep -E "DRC|violation|ANT-" /foss/designs/OpenROAD-flow-scripts/flow/logs/gf180/a
s/base/5_2_route.log | tail -10
Completing 60% with 8 violations.
Completing 70% with 8 violations.
Completing 80% with 8 violations.
Completing 90% with 8 violations.
Completing 100% with 0 violations.
[INFO DRT-0199] Number of violations = 0.
[INFO ANT-0002] Found 0 net violations.
[INFO ANT-0001] Found 0 pin violations.
[INFO ANT-0002] Found 0 net violations.
[INFO ANT-0001] Found 0 pin violations.

```

Figure 13: Example of a successful AES run using the GF180MCU OSU standard-cell library, showing the generated final output files in the OpenROAD results directory.

In Jupyter mode, interactive layout inspection is less convenient because Jupyter primarily provides a terminal-based environment rather than a full graphical desktop interface. For this reason, if the goal is to inspect layout layers visually, there are two practical options. The first option is to download the generated GDS file from Jupyter and open it locally in KLayout. The second option is to use the VNC interface and view the layout directly inside the container.

If the VNC method is preferred, open a Windows Command Prompt and run:

```

cd %USERPROFILE%\eda\designs\iic-osic-tools
SET DOCKER_TAG=chipathon26
start_vnc.bat

```

After the VNC container starts, open Docker Desktop and select the corresponding VNC port, as shown in Figure 14. Then open the VNC session in a web browser. When the login prompt

appears, enter the default password:

abc123

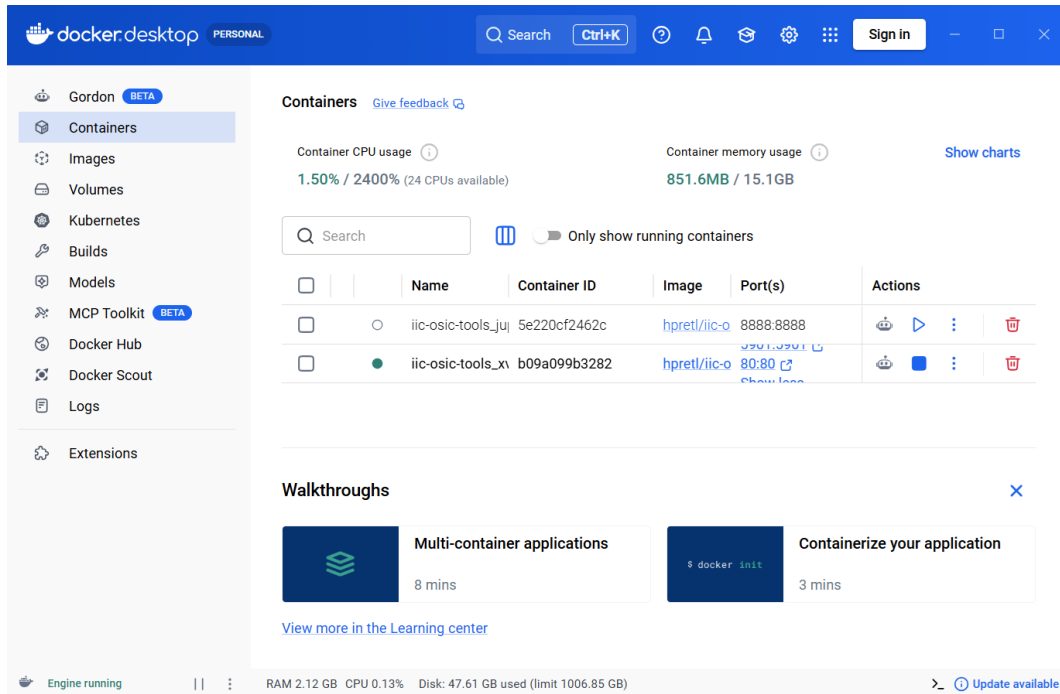


Figure 14: Docker Desktop showing the running VNC container and the port used to open the graphical desktop session.

Once the graphical desktop opens, right-click on the desktop background and open a terminal window inside the VNC session.



Figure 15: Opening a terminal in the VNC session.

Then launch KLayout with:

```
klayout /foss/designs/OpenROAD-flow-scripts/flow/results/gf180/aes/base/6_1_merged.gds
```

A full-chip view of the complete final GDS layout is shown in Figure 16.

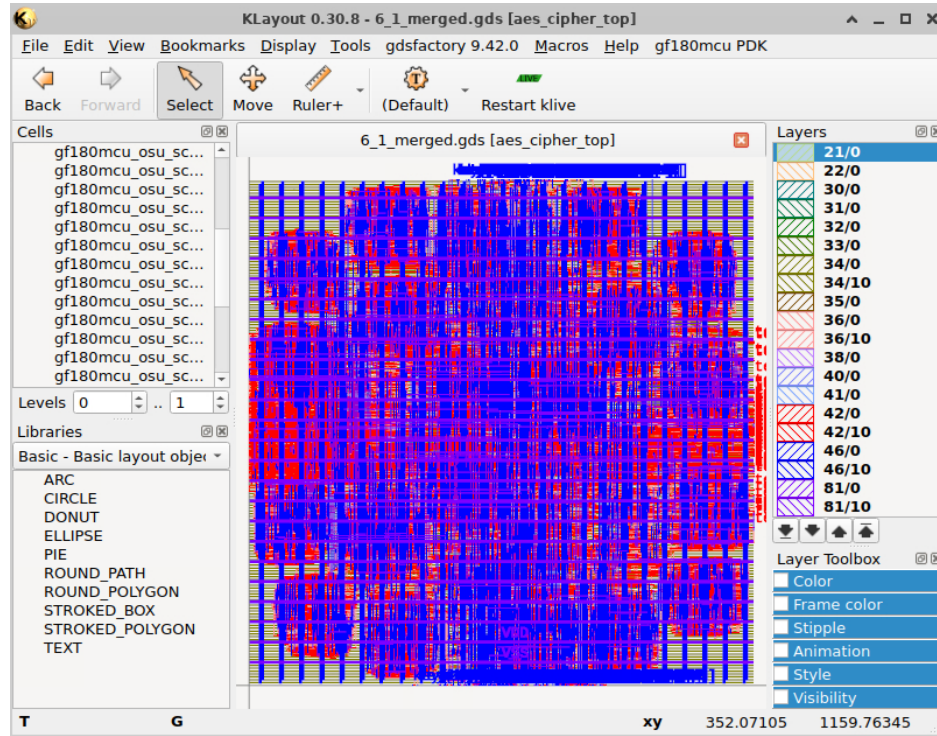


Figure 16: Top-level view of the complete final AES GDS layout generated with the GF180MCU OSU standard-cell library, shown in KLayout.

The zoomed-in view in Figure 17 shows the layout in greater detail and makes the OSU standard cells visible inside the final design.

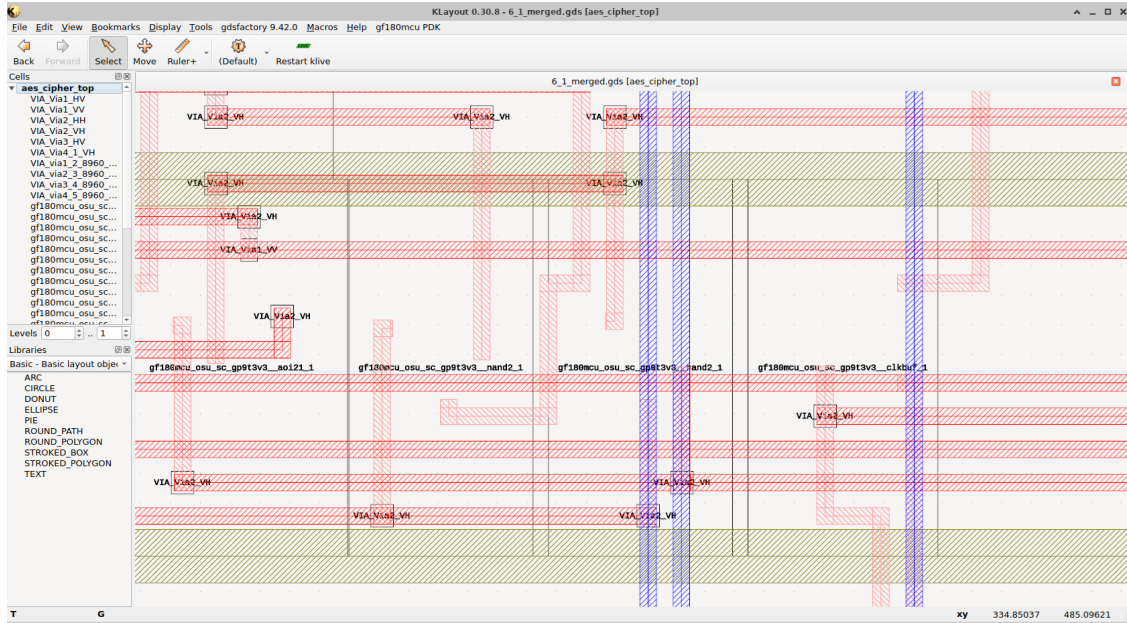


Figure 17: Zoomed-in view of the generated GDS layout in KLayout, showing the routing layers and visible OSU standard cells inside the final design.

6 Debugging and Problem Resolution

This section documents several practical issues encountered during the setup and execution of the OpenROAD workflow, together with the corresponding debugging steps and solutions. Because this workflow integrates multiple tools, scripts, libraries, and container configurations, various compatibility and runtime problems may appear during execution. Recording these issues and their corresponding solutions is important because it helps future users reproduce the workflow more reliably and understand the reasoning behind each modification.

6.1 Incorrect ABC_DRIVER_CELL and MIN_BUF_CELL Names

Problem: The original platform configuration used incorrect cell-name expressions for the buffer cells:

```
export ABC_DRIVER_CELL = gf180mcu_osu_sc_gp9t3v3$(TRACK_OPTION)$(POWER_OPTION)__buf_4
export MIN_BUF_CELL_AND_PORTS = gf180mcu_osu_sc_gp9t3v3$(TRACK_OPTION)$(POWER_OPTION)
__buf_1 A Y
```

Debugging: The correct buffer-cell names were checked from the Liberty files using:

```
grep -i "cell.*buf_4" ../lib/*.lib | head -5
```

Solution: The cell names were corrected as follows:

```
export ABC_DRIVER_CELL = gf180mcu_osu_sc_gp9t3v3__buf_4
export MIN_BUF_CELL_AND_PORTS = gf180mcu_osu_sc_gp9t3v3__buf_1 A Y
```

6.2 Broken Final Line in the Platform config.mk

Problem: The last line of the platform configuration file was corrupted:

```
-include $(GF180_PRIVATE_DIR)/private.mk80_PRIVATE_DIR)/private.mk
```

Solution: The line was corrected to:

```
-include $(GF180_PRIVATE_DIR)/private.mk
```

6.3 Resolving the SITE Definition Mismatch for OSU Library Integration

Problem: During detailed placement, the flow failed because the site definition used by the OSU standard-cell library was not consistent with the default GF180 technology LEF used in the platform setup. As a result, the placement rows generated from the platform technology definition did not match the physical site requirements of the OSU cells, which led to complete detailed-placement failure:

```
Diamond Move Success: 0 (0.00%)
Total Placement Failures: 11004
[ERROR DPL-0036] Detailed placement failed
```

Debugging: To identify the cause of the failure, the site definitions in the OSU library LEF and the default GF180 technology LEF were compared:

```
grep -A5 "^SITE" .../lef/gf180mcu_osu_sc_gp9t3v3_site.lef
grep -A5 "^SITE" .../lef/gf180mcu_5LM_1TM_9K_9t_tech.lef
```

The OSU library used the following site size:

```
SIZE 0.56 BY 6.35
```

whereas the default GF180 platform technology LEF used:

```
SIZE 0.56 BY 5.04
```

This mismatch indicated that the placement site used by the platform was incompatible with the OSU cell library in this integration setup.

Solution: Instead of modifying the original technology LEF directly, a separate adapted technology LEF file was created for the OSU-based flow. In this file, the site name and site size were updated so that the platform row definition matched the OSU library site definition:

```
cp ../gf180mcu_5LM_1TM_9K_9t_tech.lef \  
    ../gf180mcu_5LM_1TM_9K_9t_tech_osu.lef  
  
sed -i 's/SITE GF018hv5v_green_sc9/SITE gf180mcu_osu_sc_gp9t3v3/g' \  
    ../gf180mcu_5LM_1TM_9K_9t_tech_osu.lef  
  
sed -i 's/SIZE 0.56 BY 5.04/SIZE 0.56 BY 6.35/g' \  
    ../gf180mcu_5LM_1TM_9K_9t_tech_osu.lef  
  
sed -i 's/END GF018hv5v_green_sc9/END gf180mcu_osu_sc_gp9t3v3/g' \  
    ../gf180mcu_5LM_1TM_9K_9t_tech_osu.lef
```

The design configuration was then updated to use the adapted technology LEF:

```
export TECH_LEF = ../gf180mcu_5LM_1TM_9K_9t_tech_osu.lef
```

Remarks: This modification is used here as an integration step to make the OpenROAD platform technology definition consistent with the OSU library site definition. It is not presented as a universal modification to the GF180 platform, but as a library-specific compatibility adjustment for this worked example.